Dayjs

划分测试单元

1. parseDate(cfg)

原函数

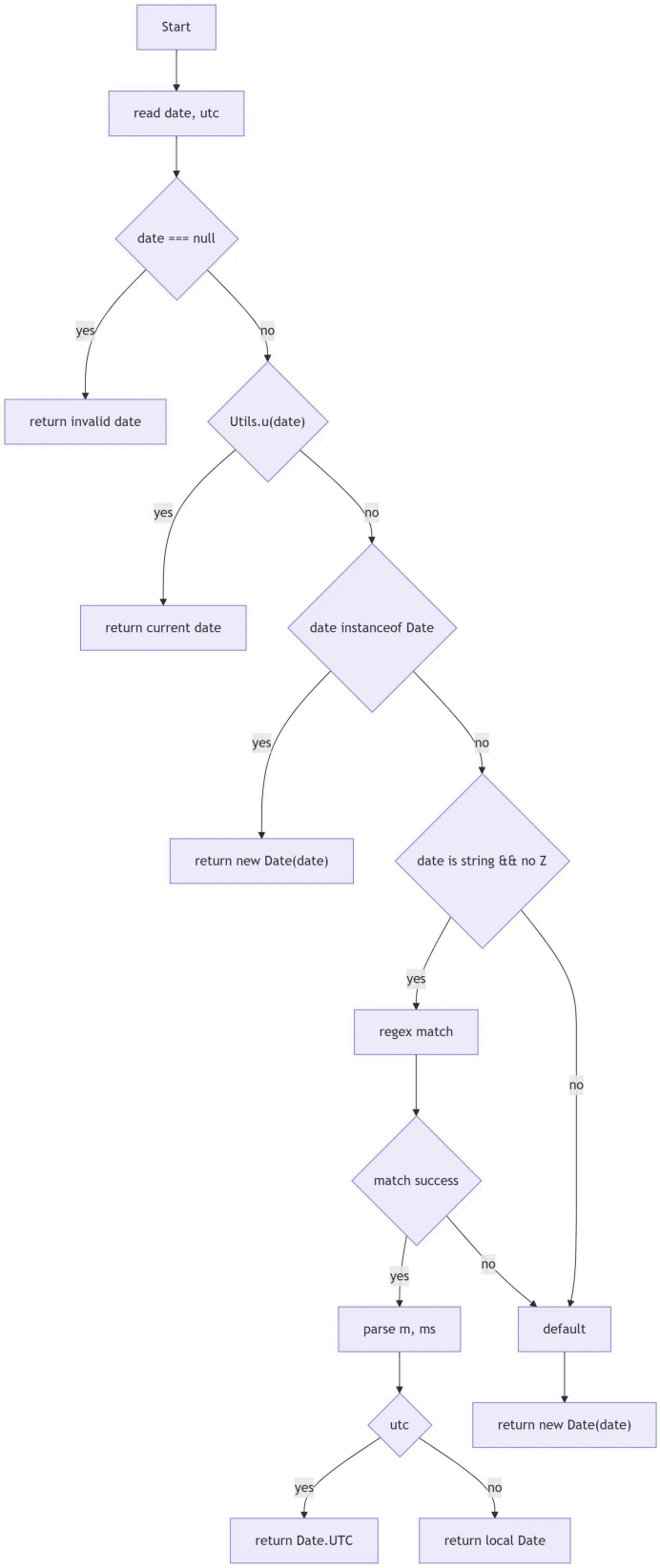
- 功能：
  - 将输入日期解析为 JavaScript Date 对象
  - 支持输入：
    i. null
    ii. undefined
    iii. Date 对象
    iv. 字符串日期
    v. 其他类型

JavaScript

```javascript
// 注释划分基本块
const parseDate = (cfg) => {
  const { date, utc } = cfg  // B1
  if (date === null)  // B2
    return new Date(NaN)  // B3
  if (Utils.u(date))  // B4
    return new Date()  // B5
  if (date instanceof Date)  // B6
    return new Date(date)  // B7

  if (typeof date === 'string' && !/Z$/i.test(date)) {  // B8
    const d = date.match(C.REGEX_PARSE)  // B9
    if (d) {  // B10
      const m = d[2] - 1 || 0  // B11
      const ms = (d[7] || '0').substring(0, 3)

      if (utc) {  // B12
        return new Date(Date.UTC(d[1], m, d[3]
          || 1, d[4] || 0, d[5] || 0, d[6] || 0, ms))  // B13
      }

```

```
22        return new Date(d[1], m, d[3]
23          || 1, d[4] || 0, d[5] || 0, d[6] || 0, ms)  // B14
24      }
25    }
26
27    return new Date(date)  // B15
28  }
```

控制流图（CFG）



圈复杂度计算

- 相关公式：
  - 圈复杂度公式： $V(G) = E - N + 2$
  - 简化公式： $V(G) = \text{判定节点数} + 1$

- 计算：
  - 判定节点：
    i. `date === null`
    ii. `Utils.u(date)`
    iii. `instanceof Date`
    iv. `string && !Z`
    v. `regex match`
    vi. `utc`
  - 因此， $V(G) = 6 + 1 = 7$,需要至少 7 条独立路径测试
    - Path1：null 输入
      - Start → date === null → return invalid date
    - Path2：undefined 输入
      - Start → date ≠ null → Utils.u(date) → return current date
    - Path3：Date 对象输入
      - Start → date ≠ null → not undefined → instanceof Date → return copy
    - Path4：字符串 + regex 匹配 + utc=true
      - Start → string → regex match → utc=true → Date.UTC
    - Path5：字符串 + regex 匹配 + utc=false
      - Start → string → regex match → utc=false → local Date
    - Path6：字符串但 regex 不匹配
      - Start → string → regex fail → fallback new Date(date)
    - Path7：非字符串输入
      - Start → not string → fallback new Date(date)

> Path 6（字符串但regex不匹配）和Path 7（非字符串）都走到`return new Date(date)`（B15），但它们经过的判定节点不同——这两条路径在语句覆盖上等价，但在**分支覆盖**上不等价

测试用例设计

Test ID	输入	覆盖路径	预期输出
TC1	null	Path1	Invalid Date
TC2	undefined	Path2	当前时间
TC3	new Date()	Path3	相同日期对象
TC4	"2024-01-01"+utc=true	Path4	UTC时间
TC5	"2024-01-01"+utc=false	Path5	本地时间
TC6	"invalid-date"	Path6	Invalid Date
TC7	1700000000000	Path7	时间戳解析

---

2. parseLocale(preset, object, isLocal)

原函数

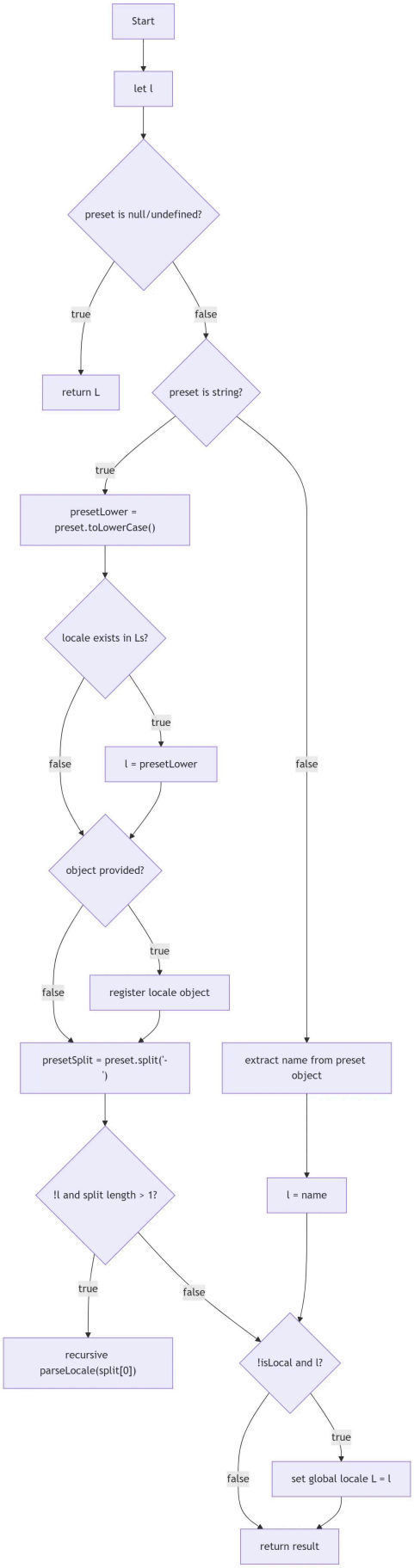
- 功能：
  - 解析并设置语言 locale 配置
- 支持输入：
  a. `null/undefined` → 返回当前默认locale
  b. `string` → 查找locale
  c. `string + object` → 注册新locale
  d. `object` → 直接注册 locale
  e. `xx-yy` → fallback到 `xx`

JavaScript

```javascript
// 注释划分基本块
const parseLocale = (preset, object, isLocal) => {
  let l  // B1
  if (!preset)  // B2
    return L  // B3
  if (typeof preset === 'string') {  // B4
    const presetLower = preset.toLowerCase()  // B5
    if (Ls[presetLower]) {  // B6
```

```
9         l = presetLower  // B7
10      }
11      if (object) {  // B8
12        Ls[presetLower] = object  // B9
13        l = presetLower
14      }
15      const presetSplit = preset.split('-')  // B10
16      if (!l && presetSplit.length > 1) {  // B11
17        return parseLocale(presetSplit[0])  // B12
18      }
19    } else {  // B13
20      const { name } = preset  // B14
21      Ls[name] = preset
22      l = name
23    }
24    if (!isLocal && l)  // B15
25      L = l  // B16
26    return l || (!isLocal && L)  // B17
27  }
```

控制流图（CFG）


Start
let l
preset is null/undefined?
true
false
return L
preset is string?
true
presetLower = preset.toLowerCase()
locale exists in Ls?
true
false
l = presetLower
false
object provided?
true
false
register locale object
presetSplit = preset.split('-')
extract name from preset object
!l and split length > 1?
l = name
true
false
recursive parseLocale(split[0])
!isLocal and l?
true
false
set global locale L = l
return result

圈复杂度计算

- 相关公式：

- 圈复杂度公式： $V(G) = E - N + 2$
- 简化公式： $V(G) = \text{判定节点数} + 1$

- 计算：
  - 判定节点：
    i. `if(!preset)`
    ii. `if(typeof preset === 'string')`
    iii. `if(Ls[presetLower])`
    iv. `if(object)`
    v. `if(!l && presetSplit.length > 1)`
    vi. `if(!isLocal && l)`
  - 因此， $V(G) = 6 + 1 = 7$,需要至少 7 条独立路径测试
    - Path1：preset 为空
      - B1 → B2(true) → B3
    - Path2：字符串 locale 已存在
      - B1 → B2(false)
        - → B4(true)
        - → B5
        - → B6(true)
        - → B7
        - → B8(false)
        - → B10
        - → B11(false)
        - → B15(true)
        - → B16
        - → B17
    - Path3：字符串 locale 不存在
      - B1 → B2(false)

        - → B4(true)
        - → B5
        - → B6(false)
        - → B8(false)
        - → B10
        - → B11(false)
        - → B15(false)
        - → B17
    - Path4：字符串 + object 注册新 locale
        - B1 → B2(false)
            - → B4(true)
            - → B5
            - → B6(false)
            - → B8(true)
            - → B9
            - → B10
            - → B11(false)
            - → B15(true)
            - → B16
            - → B17
    - Path5：locale fallback 机制
        - B1 → B2(false)
            - → B4(true)
            - → B5
            - → B6(false)
            - → B8(false)
            - → B10

        - → B11(true)
        - → B12
    - Path6：object 形式 locale
        - B1 → B2(false)
            - → B4(false)
            - → B13
            - → B14
            - → B15(true)
            - → B16
            - → B17
    - Path7：isLocal = true
        - B1 → B2(false)
            - → B4(true)
            - → B6(true)
            - → B8(false)
            - → B10
            - → B11(false)
            - → B15(false)
            - → B17

测试用例设计

Test ID	输入	覆盖路径	预期输出
TC1	parseLocale(null)	Path1	返回默认locale
TC2	parseLocale("en")	Path2	返回en并更新全局
TC3	parseLocale("abc")	Path3	返回默认locale

TC4	parseLocale("fr", {...})	Path4	Ls["fr"]被创建
TC5	parseLocale("en-US")	Path5	返回en
TC6	parseLocale({name:"jp"})	Path6	注册jp
TC7	parseLocale("en",null,true)	Path7	不修改全局

> **TC3 执行轨迹分析（`parseLocale("abc")`）：**
>
> 1. `!preset` → false，继续执行
> 2. `typeof preset === 'string'` → true，进入字符串分支
> 3. `presetLower = "abc"`
> 4. `Ls["abc"]` 不存在 → `l` 保持 undefined
> 5. `object` 为 null/undefined → 跳过注册
> 6. `presetSplit = ["abc"]`，长度为1
> 7. `!l && presetSplit.length > 1` → `true && false` → **false**，不触发fallback递归
> 8. `!isLocal && l` → `l` 为 undefined → false，不更新全局L
> 9. `return l || (!isLocal && L)` → `return undefined || L` → **返回当前全局默认locale**
>
> 注意：此路径与TC5（`"en-US"` fallback）的关键区别在于第7步——`"abc"`没有连字符导致`presetSplit.length === 1`，所以**不会**触发fallback递归，而是直接静默返回默认值。这是一个容易被AI忽略的细节：AI可能将"未知locale"和"带连字符的未知locale"归为同一等价类，从而漏掉TC5的fallback路径。

---

3. Dayjs.prototype.diff(input, units, float)

原函数

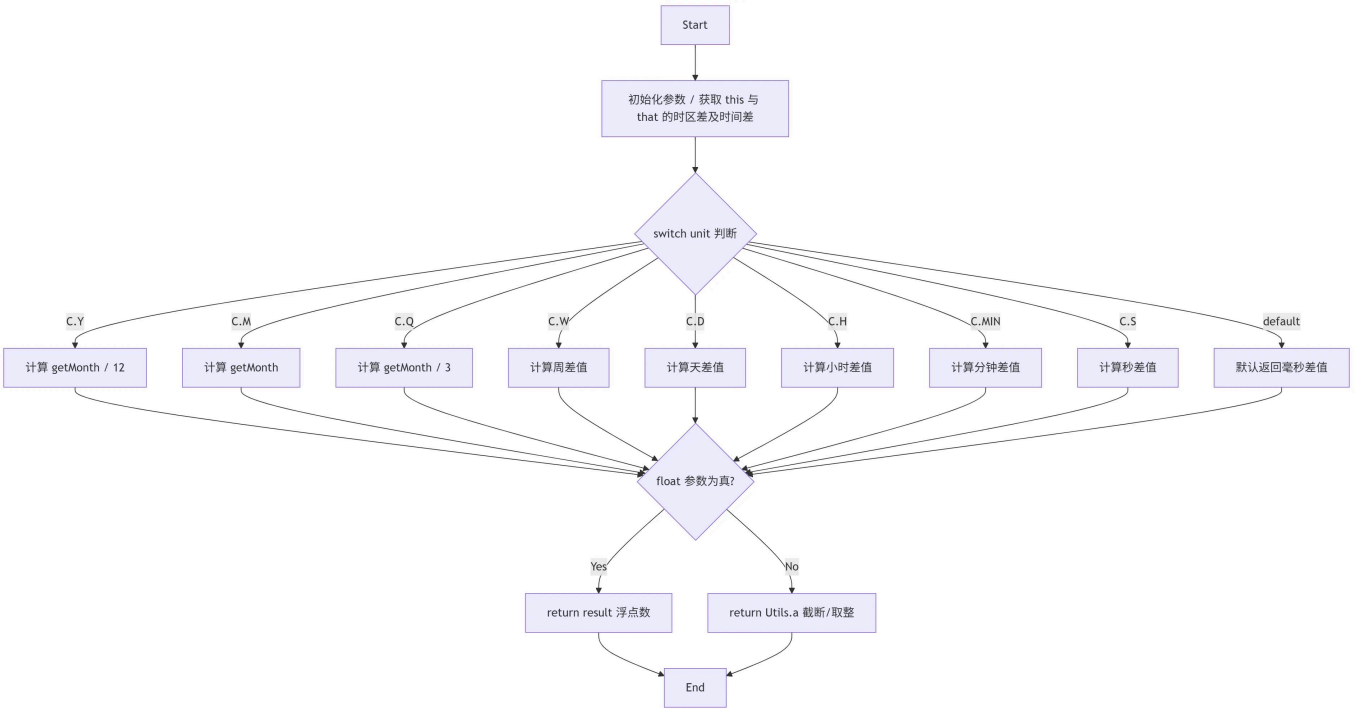
- 功能：
  - 计算两个 Dayjs 时间对象之间的差值。

JavaScript

```javascript
// 注释划分基本块
diff(input, units, float) {
    const unit = Utils.p(units)  // B1
```

```
    const that = dayjs(input)
    const zoneDelta = (that.utcOffset() - this.utcOffset()) * C.MILLISECONDS_A_MINUTE
    const diff = this - that
    const getMonth = () => Utils.m(this, that)

    let result
    switch (unit) {  // B2
      case C.Y:  // B3
        result = getMonth() / 12
        break
      case C.M:  // B4
        result = getMonth()
        break
      case C.Q:  // B5
        result = getMonth() / 3
        break
      case C.W:  // B6
        result = (diff - zoneDelta) / C.MILLISECONDS_A_WEEK
        break
      case C.D:  // B7
        result = (diff - zoneDelta) / C.MILLISECONDS_A_DAY
        break
      case C.H:  // B8
        result = diff / C.MILLISECONDS_A_HOUR
        break
      case C.MIN:  // B9
        result = diff / C.MILLISECONDS_A_MINUTE
        break
      case C.S:  // B10
        result = diff / C.MILLISECONDS_A_SECOND
        break
      default:  // B11
        result = diff
        break
    }

    return float ? result : Utils.a(result)  // B12
  }
```

控制流图（CFG）



圈复杂度计算

- 相关公式：
  - 圈复杂度公式： $V(G) = E - N + 2$
  - 简化公式： $V(G) = \text{判定节点数} + 1$
- 计算：
  - 判定节点：
    i. `Y`
    ii. `M`
    iii. `Q`
    iv. `W`
    v. `D`
    vi. `H`
    vii. `MIN`
    viii. `S`
    ix. `default`
  - 因此， $V(G) = 9 + 1 = 10$,需要至少 10 条独立路径测试
    - Path1：年+浮点

        - Start → switch(Y) → float(True) → 返回浮点数
    - Path2：月+浮点
        - Start → switch(M) → float(True) → 返回浮点数
    - Path3：季度+浮点
        - Start → switch(Q) → float(True) → 返回浮点数
    - Path4：周+浮点
        - Start →switch(W)→float(True) → 返回浮点数
    - Path5：天+浮点
        - Start → switch(D) → float(True) → 返回浮点数
    - Path6：小时+浮点
        - Start → switch(H) → float(True) → 返回浮点数
    - Path7：分钟+浮点
        - Start → switch(MIN) → float(True) → 返回浮点数
    - Path8：秒+浮点
        - Start → switch(S) → float(True) → 返回浮点数
    - Path9：毫秒/默认+浮点
        - Start → switch(default) → float(True) → 返回浮点数
    - Path10：验证取整逻辑
        - Start → 走任意 switch（例如M）→ float(False) → 调用Utils.a(result) 返回整数（此路径引入了由三元运算符产生的最新独立边）

测试用例设计

> 设定：所有测试用例中，当前对象 this 为 dayjs('2024-06-01T12:00:00')

Test ID	输入	覆盖路径	预期输出
TC1	('2023-06-01', 'year', true)	Path1	1
TC2	('2024-04-15', 'month', true)	Path2	1.5

TC3	('2023-12-01', 'quarter', true)	Path3	2
TC4	('2024-05-18', 'week', true)	Path4	2
TC5	('2024-05-30', 'day', true)	Path5	2
TC6	('2024-06-01T09:30:00', 'hour', true)	Path6	2.5
TC7	('2024-06-01T11:45:00', 'minute', true)	Path7	15
TC8	('2024-06-01T11:59:30', 'second', true)	Path8	30
TC9	('2024-06-01T11:59:59.500', 'ms', true)	Path9	500
TC10	('2024-04-15', 'month', false)	Path10	1
TC11	('2024-03-01', 'month', false)	Path10	2
TC12	('2024-03-01', 'month', true)	Path10	1.548…
TC13	('2024-02-01', 'month', false)	Path10	1

TC11和TC12**必须成对出现**，原因是：

> 仅测试TC10（整数结果）无法区分`Utils.a()`是`Math.round`、`Math.floor`还是`Math.trunc`——三者对整数结果均返回相同值。TC11通过构造一个真值为1.548的差值，使三种实现产生不同结果（round→2，floor/trunc→1），从而真正验证了取整语义。TC12作为对照组，用相同输入但`float=true`返回浮点数，一方面确认浮点真值，另一方面验证`float`参数的分支逻辑确实生效。